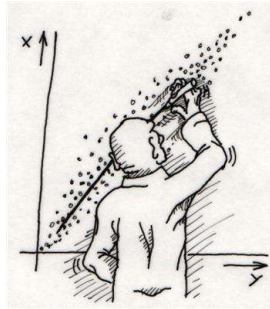


APPRENTISSAGE SUPERVISÉ

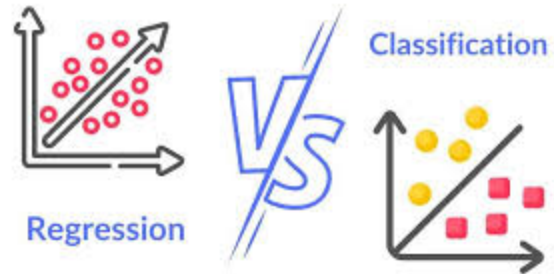
Régression



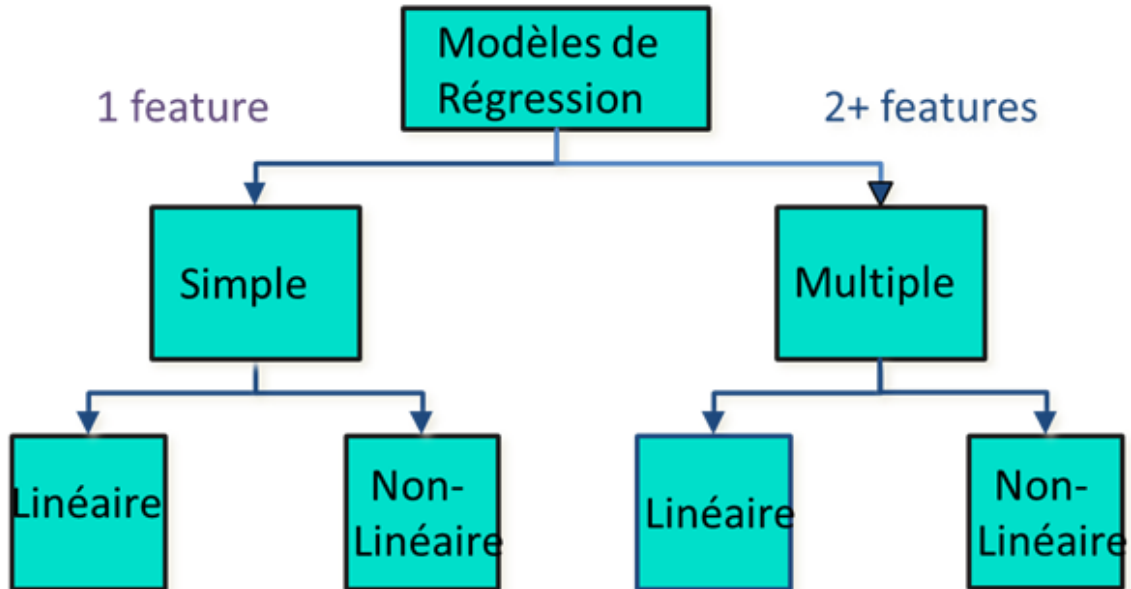
Différence entre la classification et la régression

La classification permet de prédire le résultat d'un échantillon donné pour les variables de sortie qui se présentent sous forme de catégorie.

La régression par contre est utilisée pour prédire le résultat d'un échantillon lorsque la variable de sortie est une valeur réelle spécifique.



Types des modèles de régression





Applications

La régression est utilisée dans divers domaines par exemple :

- 1 Prédire les cours boursiers
- 2 Estimer le prix de l'immobilier
- 3 Prévoir la consommation énergétique

Sortie : Le résultat d'un modèle de régression est une valeur continue, par exemple un prix, une température, ou un taux de croissance $\implies$ La régression est le processus de prédiction d'une valeur continue.

Exemple de problème de régression

Estimer le prix d'une maison en fonction de sa superficie, du nombre des étages, du nombre des chambres et de l'année de construction.

La régression est supervisée : la cible est fournie.

X: variable indépendante

Y: Variable dépendante

Surface (m ²)	Nombre de chambres	Nombre d'étages	Âge de la maison (années) x	Prix (1000\$)
200	5	1	45	460
130	3	2	40	232
140	3	2	30	315
80	2	1	36	178
150	3	2	30	?

Variable continue





Définition de la régression

La régression linéaire est une méthode d'analyse qui date de plusieurs centaines d'années.



Définition de la régression

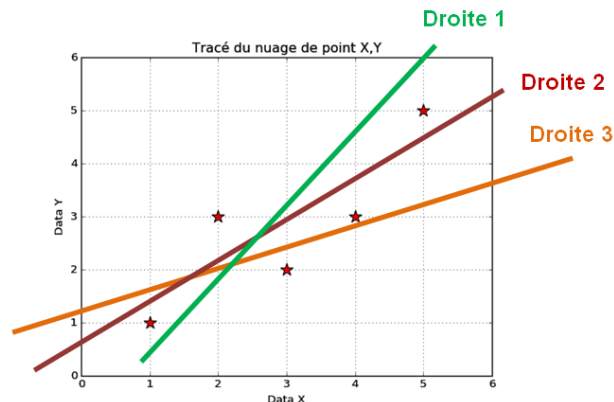
La régression linéaire est une méthode d'analyse qui date de plusieurs centaines d'années.

C'est une technique utilisée pour modéliser et analyser la relation entre une variable dépendante (y) et une ou plusieurs variables indépendantes (x).

Définition de la régression

La régression linéaire est une méthode d'analyse qui date de plusieurs centaines d'années.

C'est une technique utilisée pour modéliser et analyser la relation entre une variable dépendante (y) et une ou plusieurs variables indépendantes (x). L'objectif de la régression est de prédire ou d'expliquer la valeur y (output) en fonction de x (input).

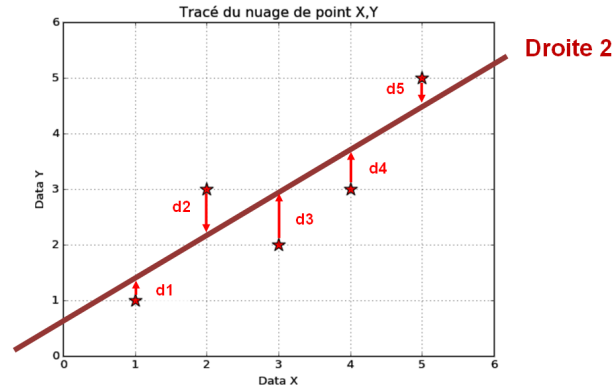




Le fonctionnement de la régression consiste à placer la droite qui se trouve la plus proche de tous les points du jeu de données c-à-d de telle manière que la distance entre tous les points du dataset et la droite soit minimale.

Le fonctionnement de la régression consiste à placer la droite qui se trouve la plus proche de tous les points du jeu de données c-à-d de telle manière que la distance entre tous les points du dataset et la droite soit minimale.

⇒ Le but est donc de minimiser l'erreur globale pour obtenir un modèle qui prédit correctement les nouvelles données.

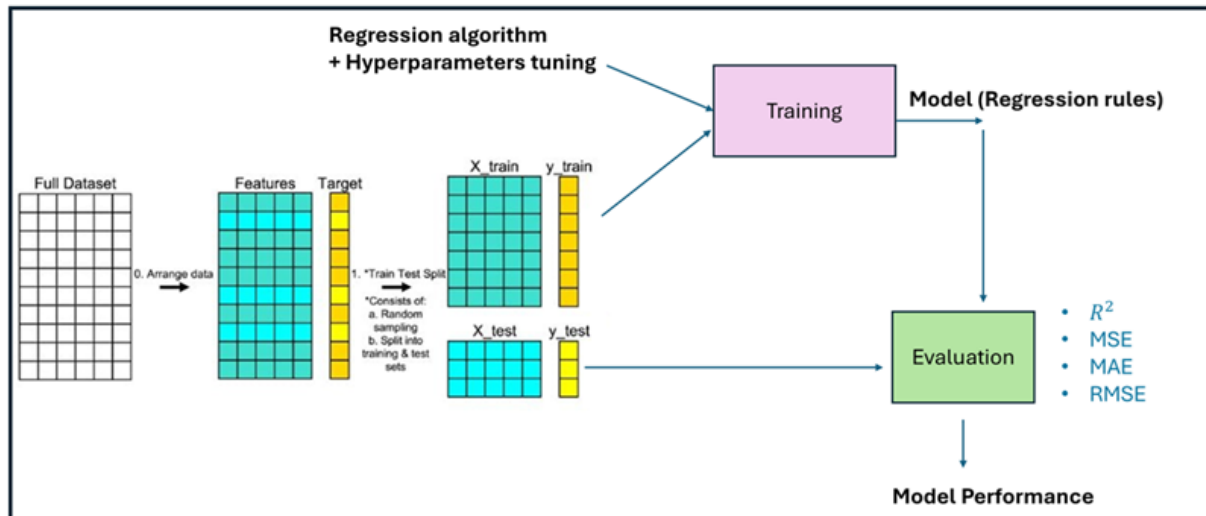




Exemples des algorithmes de régression

- ① Régression ordinale
- ② Régression de Poisson
- ③ Régression quantile rapide par forêt aléatoire
- ④ Régressions linéaire, polynomiale, Lasso, pas à pas, Ridge
- ⑤ Régression linéaire bayésienne
- ⑥ Régression par réseau de neurones
- ⑦ Régression par forêt d'arbres de décision
- ⑧ K plus proches voisins (KNN)
- ⑨ Régression par arbre de décision renforcé (Boosted decision tree regression)

Processus de régression



Régression Linéaire

L'analyse de régression linéaire est utilisée pour créer un modèle qui décrit la relation entre une variable dépendante et une ou plusieurs variables indépendantes. Selon qu'il y a une ou plusieurs variables indépendantes, on distingue l'analyse de régression linéaire simple et l'analyse de régression linéaire multiple.

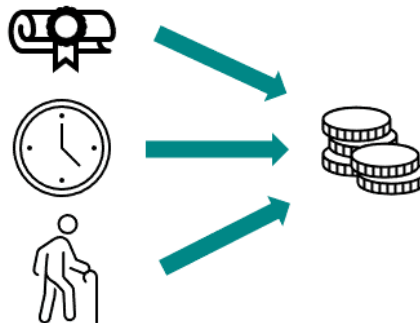
Régression Linéaire

L'analyse de régression linéaire est utilisée pour créer un modèle qui décrit la relation entre une variable dépendante et une ou plusieurs variables indépendantes. Selon qu'il y a une ou plusieurs variables indépendantes, on distingue l'analyse de régression linéaire simple et l'analyse de régression linéaire multiple.

Simple Linear Regression



Multiple Linear Regression



Exemple : Simple features (une seule variable)

Surface (m ²)	Prix (en milliers €)
200	460
130	232
140	315
80	178
150	?

L'équation de la droite de régression
est : $h_{\theta}(x) = \theta_0 + \theta_1 x$.

Exemple : Exemple: Multiple features (plusieurs variables)

Surface x_1	Nombre de chambres x_2	Nombre d'étages x_3	Âge de la maison (années) x_4	Prix (MD) y
200	5	1	45	460
130	3	2	40	232
140	3	2	30	315
80	2	1	36	178
150	3	2	30	??

L'équation de la droite de régression
est : $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$.

Exemple : Exemple: Multiple features (plusieurs variables)

Surface x_1	Nombre de chambres x_2	Nombre d'étages x_3	Âge de la maison (années) x_4	Prix (MD) y
200	5	1	45	460
130	3	2	40	232
140	3	2	30	315
80	2	1	36	178
150	3	2	30	??

L'équation de la droite de régression
est : $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$.

Dans le cas général, on a : $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$.

On pose $X = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$ et $\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{pmatrix} \implies h_{\theta}(x) = \theta^T X$.

Introduction

Pour construire le meilleur modèle de régression linéaire, il y a trois étapes clés :

Introduction

Pour construire le meilleur modèle de régression linéaire, il y a trois étapes clés :

- 1 Commencer par définir une fonction coût : c'est une fonction mathématique qui mesure l'erreur que nous commettons en approximant les données. Nous parlons aussi d'erreur induite par la modélisation.

Introduction

Pour construire le meilleur modèle de régression linéaire, il y a trois étapes clés :

- ① Commencer par définir une fonction coût : c'est une fonction mathématique qui mesure l'erreur que nous commettons en approximant les données. Nous parlons aussi d'erreur induite par la modélisation.
- ② Minimiser cette fonction coût : il faut trouver les bons paramètres de notre modèle pour minimiser l'erreur de modélisation.

Introduction

Pour construire le meilleur modèle de régression linéaire, il y a trois étapes clés :

- ① Commencer par définir une fonction coût : c'est une fonction mathématique qui mesure l'erreur que nous commettons en approximant les données. Nous parlons aussi d'erreur induite par la modélisation.
- ② Minimiser cette fonction coût : il faut trouver les bons paramètres de notre modèle pour minimiser l'erreur de modélisation.
- ③ Choisir une méthode de résolution du problème. Il existe deux méthodes :
 - une méthode de résolution numérique, la descente de gradient,
 - une méthode analytique, la méthode des moindres carrés.

Première étape

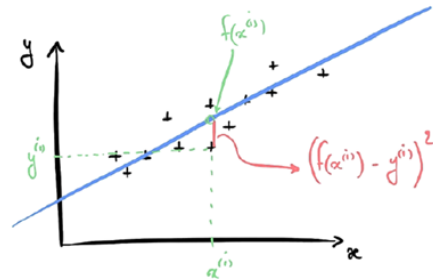
On définit la fonction coût J (Loss Function) comme étant la moyenne de toutes les erreurs :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \underbrace{[h(x_i) - y_i]^2}_{\text{erreur}_i}.$$

Exemple: Imaginez que le 10^{ième} feature de votre Dataset est un appartement dont $x_{10} = 80 \text{ m}^2$ et un prix $y_{10} = 100 \text{ MD}$ et que notre modèle linéaire simple $h_{\theta}(x) = \theta_0 + \theta_1 x$ prédise un prix de $h(x_{10}) = 100,002 \text{ MD}$.

Alors, l'erreur pour cet exemple est :

$$\text{erreur}_{10} = [h(x_{10}) - y_{10}]^2 = (100,002 - 100)^2 = 4 \cdot 10^{-6}$$



Donc chaque prédiction s'accompagne d'une erreur $\implies$ On a donc m erreurs.

Deuxième étape

On a :

$$h_{\theta}(x) = \theta^T X = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [h(x_i) - y_i]^2$$

Les paramètres : $\theta_0, \theta_1, \theta_2, \dots, \theta_n$.

$\implies$ Déterminer θ qui minimise J .

Regression linéaire simple

- Un ensemble d'entraînement $D = (X^i, y_i)_{i=1, \dots, m}$.
- Sélection du modèle : $h_\theta(x) = \theta_0 + \theta_1 x$.
- Paramètres : θ_0, θ_1 .
- Critère : Choisir ces deux paramètres de telle sorte que $h(x_i)$ soit proche de y_i pour notre ensemble de données d'entraînement c.à.d la fonction J soit minimale.
- Pour cela, on considère la fonction coût suivante :

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m [\theta_0 + \theta_1 x_i - y_i]^2.$$

Le gradient de J est :

$$\nabla J(\theta_0, \theta_1) = \begin{pmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^m [\theta_0 + \theta_1 x_i - y_i] \\ \sum_{i=1}^m [\theta_0 + \theta_1 x_i - y_i] x_i \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Régression linéaire multiple

- Matrice de conception :

$$X = \begin{pmatrix} 1 & x_1^1 & \dots & x_n^1 \\ 1 & x_1^2 & \dots & x_n^2 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^m & \dots & x_n^m \end{pmatrix}$$

Chaque ligne représente les valeurs d'entrée d'un exemple d'entraînement.

- Vecteur cible :

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

Ce vecteur contient tout les valeurs cibles

Ainsi

$$X\theta - y = \begin{pmatrix} h_{\theta}(x_1) - y_1 \\ h_{\theta}(x_2) - y_2 \\ \vdots \\ h_{\theta}(x_m) - y_m \end{pmatrix}$$

Donc, on a :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [h_{\theta}(x_i) - y_i]^2 = \frac{1}{2m} (X\theta - y)^T (X\theta - y).$$

Par la suite :

$$\nabla J(\theta) = \frac{1}{m} X^T (X\theta - y) = 0 \quad \implies \quad \theta = (X^T X)^{-1} X^T y.$$

Cette expression est connue comme la solution de l'équation normale du problème des moindres carrés.

Exercice

Soit le tableau suivant :

A	B	y
1	1	2.3
1	2	1.4
3	1	0.7
2	2	?

- ① Déterminer le vecteur des coefficients de l'équation normale.
- ② Donner l'équation de l'hyperplan de la régression linéaire.

Correction :

Soit l'équation de la droite de régression :

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2.$$

avec :

$$X = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 3 & 1 \end{pmatrix}, \quad y = \begin{pmatrix} 2.3 \\ 1.4 \\ 0.7 \end{pmatrix}.$$

On obtient donc :

$$\theta = \begin{pmatrix} 4 \\ -0.8 \\ -0.9 \end{pmatrix}.$$

Alors :

$$h_{\theta}(x) = 4 - 0.8x_1 - 0.9x_2.$$

Avantages et Inconvénients de l'Équation Normale en Machine Learning

Avantages :

- ① Solution analytique directe, simplifiant l'entraînement.
- ② Formulation intuitive et facile à implémenter.
- ③ Pas de taux d'apprentissage requis.
- ④ Fournit des solutions exactes si $(X^T X)$ est inversible.
- ⑤ Efficace pour les données de taille petite à moyenne.

Inconvénients :

- ① Complexité Computationnelle : Le calcul de $(X^T X)$ est coûteux en temps pour de grandes matrices (complexité de $O(n^3)$).
- ② Problèmes de Singularité : Nécessite que $(X^T X)$ soit inversible, avec un risque de multicollinéarité.
- ③ Sensibilité aux Valeurs Extrêmes : Fort impact des valeurs aberrantes.
- ④ Scalabilité : Moins adapté aux très grands ensembles de données.
- ⑤ Limitation aux Modèles Linéaires : Ne convient pas aux modèles non linéaires.

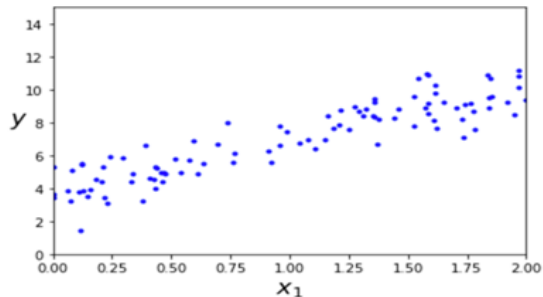
Exercice : Régression linéaire à l'aide de l'équation normale

* Générons des données d'apparence linéaire

```
import numpy as np
import matplotlib.pyplot as plt

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```

```
plt.plot(X, y, "b.")
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.axis([0, 2, 0, 15])
plt.show()
```



* Entraînement

```
X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

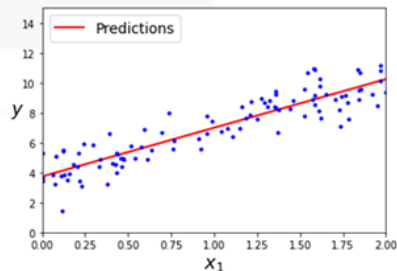
```
theta_best
```

```
array([[ 3.73953954],
       [ 3.25407465]])
```

* Faisons des prédictions

```
X_new = np.array([[0], [2]])
X_new_b = np.c_[np.ones((2, 1)), X_new] # add x0 = 1 to each instance
y_predict = X_new_b.dot(theta_best)
y_predict
```

```
array([[ 3.73953954],
       [10.24768885]])
```



* Avec Scikit-Learn

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)
lin_reg.intercept_, lin_reg.coef_

(array([3.73953954]), array([[3.25407465]]))
```

* Comment fonctionne la classe LinearRegression de sklearn ?

```
theta_best_svd, _, _, _ = np.linalg.lstsq(X_b, y, rcond=1e-6)
theta_best_svd

array([[3.73953954],
       [3.25407465]])
```


Troisième étape

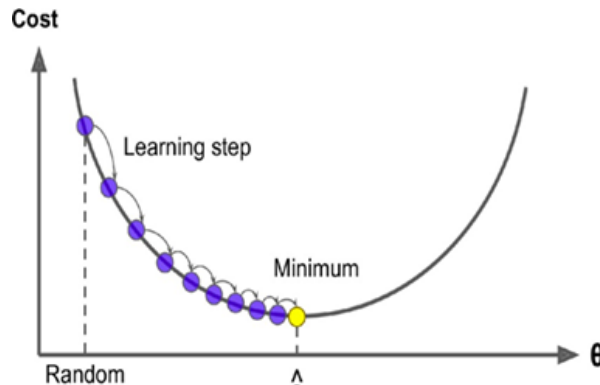
Algorithme de descente de gradient

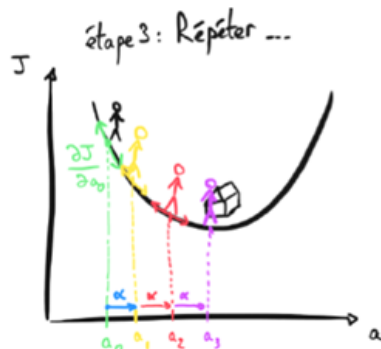
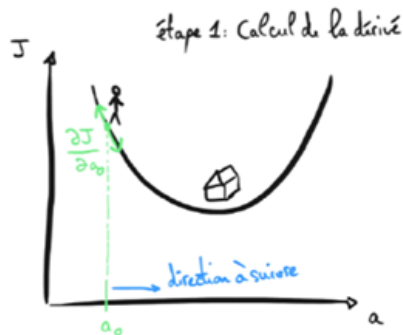
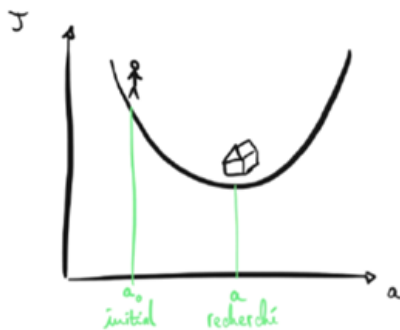
Définition :

L'algorithme de descente de gradient est une méthode d'optimisation qui consiste à mettre à jour des paramètres dans la direction opposée au gradient d'une fonction coût, afin de trouver un minimum.

L'idée principale est :

Avancer petit à petit dans la direction opposée au gradient, car c'est la direction de la plus forte diminution de la fonction coût.





Formule générale

Pour un paramètre θ , la mise à jour est donnée par :

$$\theta := \theta - \alpha \cdot \nabla_{\theta} J(\theta)$$

où :

- $J(\theta)$: la fonction coût donnée par

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m [h_{\theta}(x_i) - y_i]^2 = \frac{1}{2m} (X\theta - y)^T (X\theta - y).$$

- $\nabla_{\theta} J(\theta)$: sone gradient donné par

$$\nabla J(\theta) = \frac{1}{m} X^T (X\theta - y)$$

- α : le taux d'apprentissage (learning rate).

Exercice :

Soit le tableau utilisé précédemment :

A	B	y
1	1	2.3
1	2	1.4
3	1	0.7
2	2	?

En utilisant l'algorithme du descent du gradient avec un vecteur initial $\theta^{(0)} = (1, 1, 1)$ et un pas de descente $\alpha = 0.1$, calculer $\theta^{(1)}$ et $\theta^{(2)}$.

Correction :

On a :

$$X = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 3 & 1 \end{pmatrix}, \quad y = \begin{pmatrix} 2.3 \\ 1.4 \\ 0.7 \end{pmatrix}.$$

$$\theta^{(0)} = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}, \quad m = 3, \quad \alpha = 0.1$$

On utilise la mise à jour :

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)}),$$

Itération 1

On trouve

$$\theta^{(1)} = \begin{pmatrix} 0.74666667 \\ 0.46 \\ 0.66 \end{pmatrix}$$

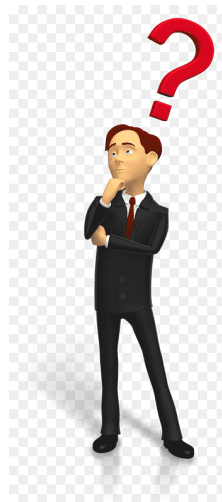
Itération 2

On trouve

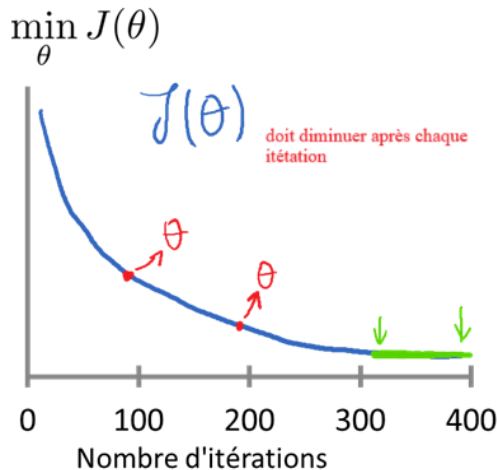
$$\theta^{(2)} = \begin{pmatrix} 0.654 \\ 0.22822222 \\ 0.52977778 \end{pmatrix}$$

Deux questions qui se posent dont on va répondre dans les prochains slides :

- 1 Comment s'assurer que la descente de gradient fonctionne correctement ?
- 2 Comment choisir le taux d'apprentissage α ?



S'assurer que la descente de gradient fonctionne correctement

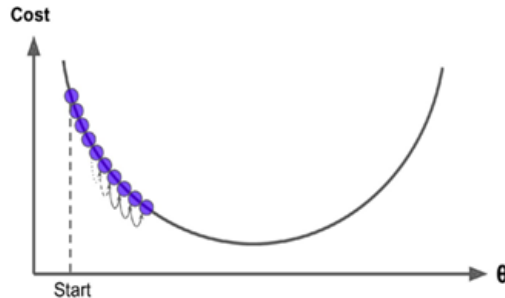


Pour des valeurs suffisamment petites, $J(\theta)$ doit diminuer à chaque itération.

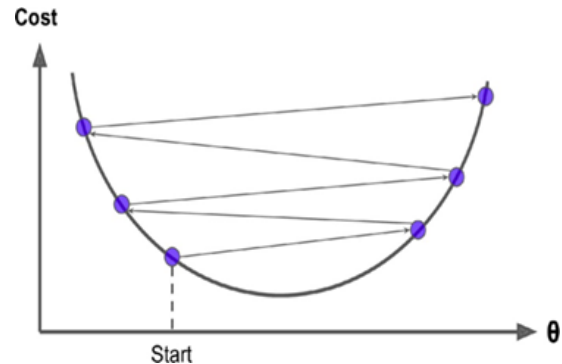
Exemple de test de convergence automatique :

Déclarer la convergence si $J(\theta)$ diminue de moins de 10^{-3} en une seule itération.

- Si α est trop petite, la descente de gradient peut être lent à converger.



- Si le taux d'apprentissage est trop grand : échec à trouver une bonne solution.



Pour des valeurs de α suffisamment petites, $J(\theta)$ doit diminuer à chaque itération.



Descente de Gradient : Avantages et Inconvénients

Avantages :

- 1) **Évolutivité** : Idéale pour les grands ensembles de données, car elle évite le calcul de l'inverse de matrices.
- 2) **Adaptée aux Modèles Complexes** : Capable de minimiser des fonctions de coût non linéaires, ce qui permet de travailler avec des modèles sophistiqués comme les réseaux de neurones.
- 3) **Contrôle de la Vitesse de Convergence** : Le taux d'apprentissage permet de moduler la vitesse de convergence.

Inconvénients :

- 1) **Paramétrage Sensible** : La descente de gradient nécessite un réglage précis du taux d'apprentissage. Un mauvais choix peut entraîner une convergence lente ou une divergence.
- 2) **Convergence à des Minima Locaux** : Pour les fonctions de coût non convexes, la descente de gradient peut se bloquer dans des minima locaux.
- 3) **Calcul Itératif** : La descente de gradient peut être lente, surtout pour des modèles avec de nombreux paramètres.

Evaluation des modèles de régression (métriques)

1) L'erreur absolue moyenne (Mean Absolute Error): MAE

Cette valeur mesure l'ampleur moyenne des erreurs entre les valeurs prédites et les valeurs réelles, sans tenir compte de la direction de ces erreurs.

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

2) L'erreur quadratique moyenne (Mean Squared Error): MSE

Cette valeur est déterminée en calculant la moyenne des différences au carré entre les valeurs réelles et les valeurs prédites.

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

3) La racine de l'erreur quadratique moyenne (Root Mean Squared Error): RMSE

Cette valeur est déterminée en calculant la racine carrée de la moyenne des différences au carré entre les valeurs réelles et les valeurs prédites.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}.$$

4) le R-carré (R squared): R^2

Cette valeur mesure la proximité des données par rapport à la ligne de régression ajustée. Il est également connu sous le nom de coefficient de détermination, ou de coefficient de détermination multiple pour la régression multiple.

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Les valeurs de R^2 sont entre 0 et 1 dans le contexte de la régression linéaire :

- ▶ 0 indique que le modèle n'explique aucune des variabilités des données de réponse autour de sa moyenne.
- ▶ 1 indique que le modèle explique toute la variabilité des données de réponse autour de sa moyenne.

Cependant, dans certains cas particuliers, comme les modèles de régression non linéaire ou les modèles mal spécifiés, les valeurs de R^2 peuvent être négatives.

Derrière chaque avancée en IA, se cache une équation bien résolue.

Merci de votre
attention!

